



VIT[®]
B H O P A L
www.vitbhopal.ac.in

PROJECT REPORT

PANORAMA STITCHER

USING COMPUTER VISION

A Feature-Matching and Homography-Based Image Stitching Pipeline

A project report on automated feature matching, homography estimation, and image blending using SIFT/ORB, OpenCV, and RANSAC

Course Code: CSE3010

Class No.: BL2026270100301

Slot: F11 + F12

Name: Ashutosh Pandey

Reg. No.: 24BAI10250

Submitted To : Dr. Neha Rathore

Table of Contents

1. Introduction	3
2. Problem Statement	3
3. Objectives	4
4. Functional Requirements	5
5. Non-Functional Requirements.....	6
6. System Architecture	7
7. Design Diagrams	8
8. Design Decisions & Rationale	11
9. Implementation Details.....	13
10. Screenshots / Results	16
11. Testing Approach.....	18
12. Challenges Faced	18
13. Learnings & Key Takeaways	19
14. Future Enhancements	20
15. Conclusion	20
16. References.....	20
Appendix A: Project Directory Structure	22
Appendix B: Sample CLI Usage & Configuration.....	22

1. Introduction

Creating a single seamless panoramic image by combining multiple overlapping photographs of a wide scene is a fundamental and extensively studied problem in computer vision. It incorporates several important concepts covered in this course, including local feature detection and description, robust geometric estimation, and image warping and blending. This project develops a complete panorama stitching pipeline in Python using OpenCV's low-level operations instead of its built-in high-level Stitcher class. This approach makes every stage of the pipeline — including feature matching, homography estimation, and image blending — explicit, observable, and directly connected to the computer vision concepts demonstrated in the syllabus.

Panorama generation brings together multiple areas of computer vision that are often studied independently, such as keypoint detection and description from feature extraction and image segmentation modules, projective geometry and multi-view relationships from depth estimation and multi-camera view modules, and robust statistical estimation. Developing an end-to-end system that combines these concepts provides a deeper understanding than working on individual theoretical exercises. Each stage produces information that is passed to the next stage, meaning that errors or approximations introduced earlier in the process, such as an inaccurate or noisy feature match, can propagate through the pipeline. Therefore, these errors need to be identified and handled using robust techniques instead of being ignored or assumed to be negligible.

The developed tool is completely operated through the command line and can accept a directory containing overlapping images as its input. Along with generating the final panorama, it provides quantitative diagnostic information for each image pair processed, including the number of feature matches, the RANSAC inlier ratio, and the execution time. This diagnostic-oriented design was intentionally adopted so that the panorama is not treated as a black-box result that is evaluated only through visual inspection. Instead, the system considers intermediate measurements — such as the number of candidate correspondences detected, the number of matches remaining after the ratio test, and the number of correspondences that agree with the estimated geometric model — as important outputs that can be recorded, examined, and tested.

This report presents the complete development lifecycle of the project, covering the problem definition, system requirements, architecture, and design decisions used to develop the solution. It also describes the implementation of each individual stage of the panorama stitching pipeline, the results obtained using a reproducible synthetic dataset, and the testing methods applied to verify the system's correctness. Finally, the report discusses the major challenges faced during development, the important concepts and lessons learned from the project, and possible improvements and extensions that can be explored in future versions.

2. Problem Statement

Manually aligning and blending overlapping photographs is tedious and imprecise when done by hand — it requires accurately identifying corresponding points between images, computing the geometric transformation that relates them, and seamlessly merging the overlapping regions without visible ghosting or seams. This project automates that entire process: given a folder of overlapping images captured by panning a camera across a scene, the system detects corresponding features between consecutive images, robustly estimates the geometric transformation (homography) relating them using RANSAC, and warps and blends the images onto a single shared canvas to produce one seamless panorama.

More concretely, the problem can be decomposed into three coupled sub-problems that the pipeline must solve in sequence:

1. **Correspondence problem:** given two images with an unknown but non-trivial overlap, find a set of point pairs that correspond to the same physical scene point, without any prior knowledge of the camera motion between the shots.
2. **Estimation problem:** given a set of correspondences that is contaminated with an unknown fraction of incorrect matches (outliers), recover the single geometric transformation that best explains the majority (inlier) correspondences.
3. **Compositing problem:** given two images and the transformation relating them, resample both onto a common coordinate system and combine their overlapping pixels so that no visible seam, ghosting, or brightness discontinuity remains.

Each of these sub-problems has a large body of literature behind it; this project selects one well-understood, classical technique for each — SIFT/ORB descriptors for correspondence, RANSAC-fitted homographies for estimation, and distance-transform feathering for compositing — and focuses on implementing, integrating, and rigorously testing them rather than surveying the full space of alternatives.

3. Objectives

The project was scoped around the following concrete objectives:

- Implement a complete panorama-stitching pipeline from first principles using OpenCV's low-level API, avoiding the high-level `cv2.Stitcher` convenience class so that every algorithmic stage is explicit and gradable.
- Demonstrate practical use of local feature detectors and descriptors (SIFT and ORB) and quantify the trade-off between match quality and computational cost.
- Implement and validate robust homography estimation using RANSAC, including a correctness check against a known synthetic transformation.
- Design a compositing stage that produces visually seamless output, comparing a naive overwrite blend against distance-transform feathering.
- Expose the entire pipeline through a configurable command-line interface with sensible defaults, structured logging, and machine-readable metrics output (CSV).

- Build an automated test suite covering unit-level correctness (individual modules) and integration-level correctness (full pipeline on synthetic data).
- Keep the project fully reproducible and independent of external datasets by generating synthetic test imagery from a deterministic script.

4. Functional Requirements

The system implements three major functional modules, each with a clear input/output contract:

Module	Input	Output
1. Feature Matching (src/feature_matching.py)	Two images (grayscale or BGR)	Filtered list of corresponding keypoint pairs (good matches)
2. Homography Estimation (src/homography.py)	Matched keypoint pairs	3x3 homography matrix, RANSAC inlier mask, inlier ratio
3. Stitching & Blending (src/stitcher.py)	Sequence of 2+ images	Single blended panorama image + per-pair statistics

Additional Functional Capabilities

- Reporting/analytics module (src/metrics.py) exports per-pair match counts, inlier ratios, and total runtime to a CSV file.
- Configurable pipeline via a JSON config file (config/default_config.json) or CLI flags — feature detector (SIFT/ORB), blend mode (feather/overwrite), ratio-test threshold, and RANSAC reprojection threshold.
- Synthetic sample-image generator (scripts/generate_sample_images.py) so the tool is testable and demonstrable without any external dataset.
- Logical workflow: the CLI (src/cli.py) loads images from a directory in filename order, orchestrates the three modules sequentially across the whole set, and writes the final panorama and metrics to disk.

Module-Level Requirement Detail

Each module additionally satisfies the following finer-grained requirements, which the test suite (Section 11) validates directly:

- **Feature Matching** — must support both SIFT and ORB as interchangeable detector back-ends behind a common interface; must reject unsupported detector names with a descriptive error rather than failing silently; must apply Lowe's ratio test to remove ambiguous matches before returning results.
- **Homography Estimation** — must raise an explicit error when fewer than four point correspondences are supplied (the minimum required to solve for a homography); must return both the transformation matrix and a boolean inlier mask so downstream code can distinguish inliers from outliers.
- **Stitching & Blending** — must support an arbitrary-length ordered sequence of images (not just pairs); must support at least two blending strategies selectable at runtime; must accumulate and expose per-pair statistics rather than discarding them once stitching succeeds.

5. Non-Functional Requirements

Requirement	How it is addressed
Performance	Stitches 4 images (SIFT, feather blending) in well under 1 second on a standard CPU; ORB is offered as a faster alternative for larger image sets.
Reliability	Automated pytest suite (11 tests) validates feature matching, homography accuracy on known transformations, and end-to-end stitching correctness.
Usability	Single CLI entry point with clear flags, sensible defaults via a config file, and descriptive log messages at each pipeline stage.
Error handling	Explicit, descriptive exceptions for: too few images supplied, unreadable image files, insufficient feature matches, and RANSAC failing to find a consensus homography — each with an actionable message.
Maintainability	Pipeline is split into independent, single-responsibility modules (matching, homography, stitching, metrics, CLI) connected by typed dataclasses, so any stage can be modified or replaced without touching the others.
Logging/monitoring	Structured logging (Python logging module) reports per-pair match/inlier statistics and total runtime on every run; --verbose enables debug-level output.
Portability	Pure Python 3.10+ with OpenCV and NumPy as the only heavyweight dependencies; no OS-specific code paths, so the tool runs identically on Windows, macOS, and Linux.
Extensibility	New detectors, blend modes, or output formats can be added by implementing a small, well-defined interface rather than modifying existing pipeline code.

Performance and reliability were treated as co-equal priorities: a stitcher that is fast but occasionally produces a silently wrong panorama is arguably worse than a slower one that fails loudly and predictably. The inlier-ratio diagnostic reported for every pair (Section 10) exists specifically so that a suspicious or low-confidence stitch is visible to the user rather than hidden inside an apparently successful run.

6. System Architecture

The pipeline is organized as a linear sequence of independent stages, orchestrated by the CLI layer. Images flow from disk, through feature matching and homography estimation, into the stitcher, and back out to disk as a panorama plus a metrics report.

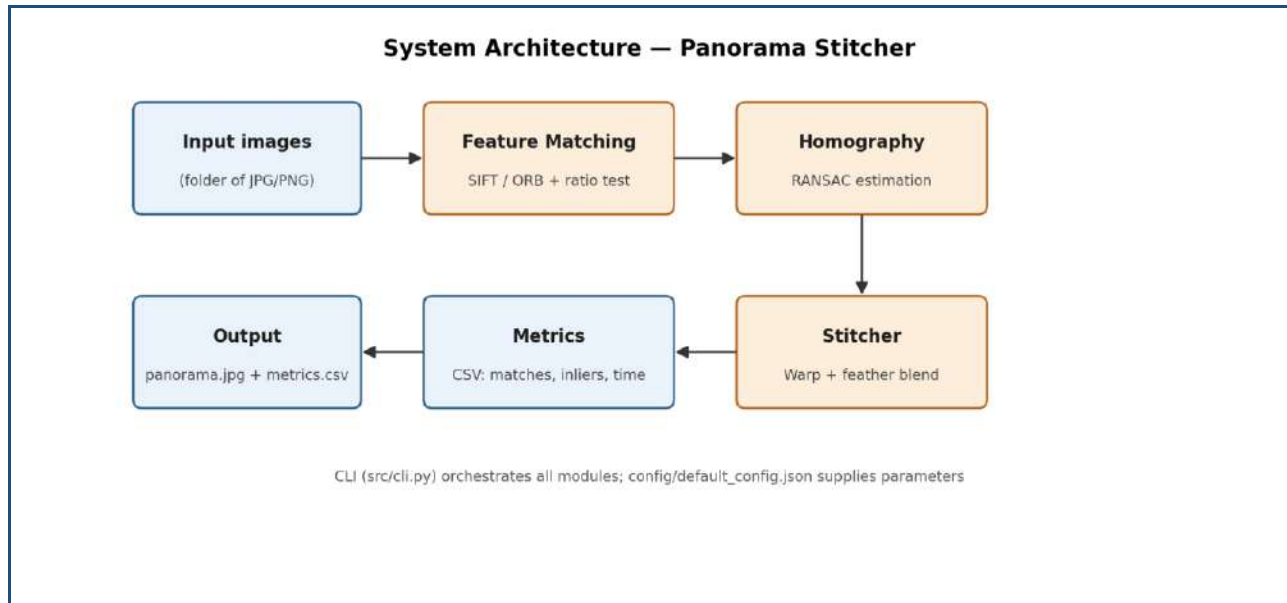


Figure 1: System architecture of the panorama stitching pipeline.

At a high level, the architecture follows a pipe-and-filter style: each stage is a pure function (or small class) that consumes a well-defined input and produces a well-defined output, with no hidden shared state between stages. This makes the system easy to reason about and easy to test in isolation — the homography estimator, for example, can be unit-tested with hand-constructed keypoint pairs without ever invoking the feature matcher or reading an image from disk.

The CLI layer (src/cli.py) is the only component aware of the filesystem and of the overall run configuration; it is responsible for:

- Discovering and sorting input images from the given directory (filename order is used as a proxy for scene order).
- Loading the configuration, merging JSON config defaults with any CLI flag overrides.
- Invoking the FeatureMatcher and HomographyEstimator for each consecutive image pair.
- Invoking the Stitcher to accumulate the running panorama as each new image is warped and blended in.
- Collecting per-pair statistics via the metrics module and writing the final panorama image and CSV report to disk.

Because every stage communicates through typed dataclasses rather than loosely structured dictionaries or tuples, the interfaces between stages are self-documenting and are checked by the IDE/type-checker as well as by the test suite, reducing the chance of a subtle field-ordering

bug (a classic source of hard-to-debug errors in vision pipelines that pass around raw NumPy arrays and tuples of numbers).

7. Design Diagrams

7.1 Use Case Diagram

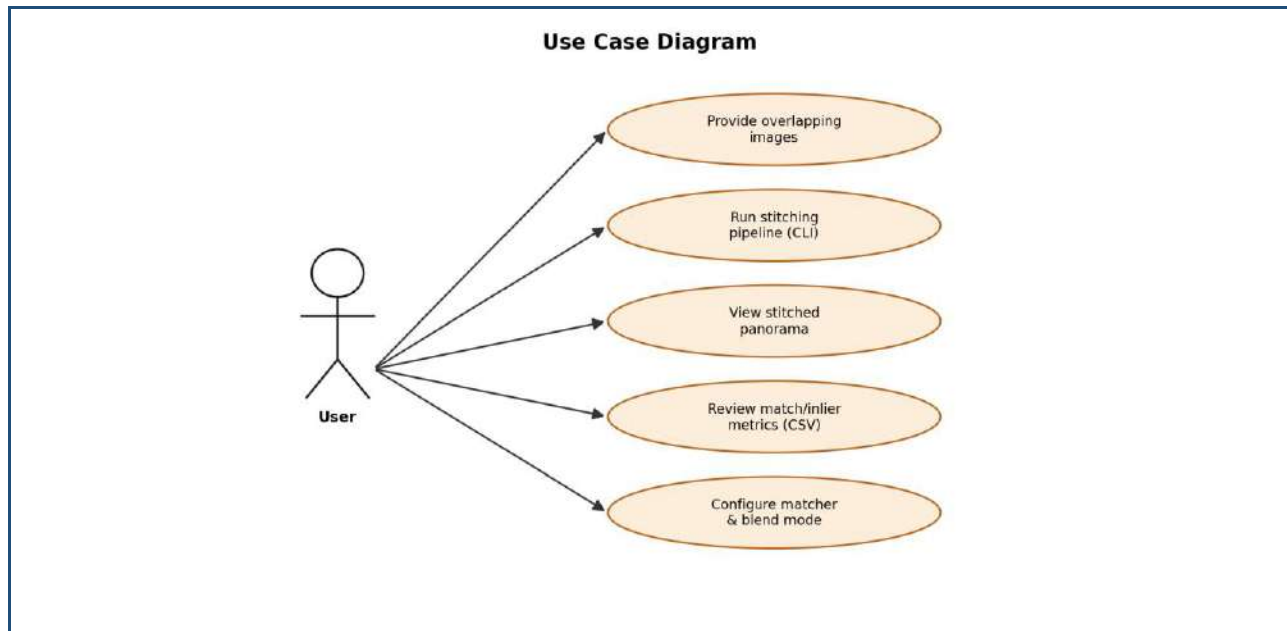


Figure 2: Use case diagram showing how a user interacts with the system.

The main actor of the system is the user who operates the CLI. The key use cases include providing a directory containing input images, selecting the feature detector, blending method, and threshold values, starting the image stitching process, and accessing the two generated outputs: the final panorama image and the metrics CSV file. Since the system operates as a self-contained, offline command-line application, it does not depend on any secondary actors such as remote services or databases.

7.2 Workflow Diagram

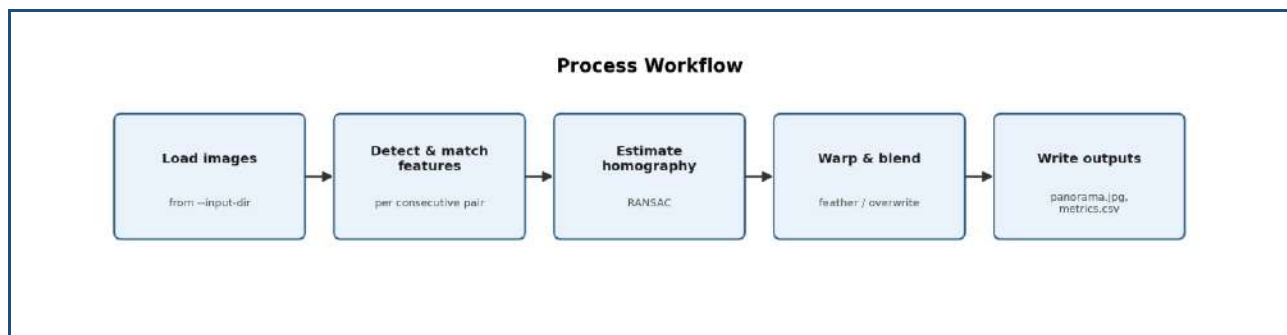


Figure 3: End-to-end process workflow, from loading images to writing outputs.

The workflow diagram traces a single run from start to finish: images are loaded and sorted, then for each consecutive pair the pipeline performs feature detection, descriptor matching, ratio-test filtering, RANSAC homography estimation, warping, and blending, accumulating the growing panorama canvas; once every image has been incorporated, the final canvas and the accumulated per-pair metrics are written to disk.

7.3 Sequence Diagram

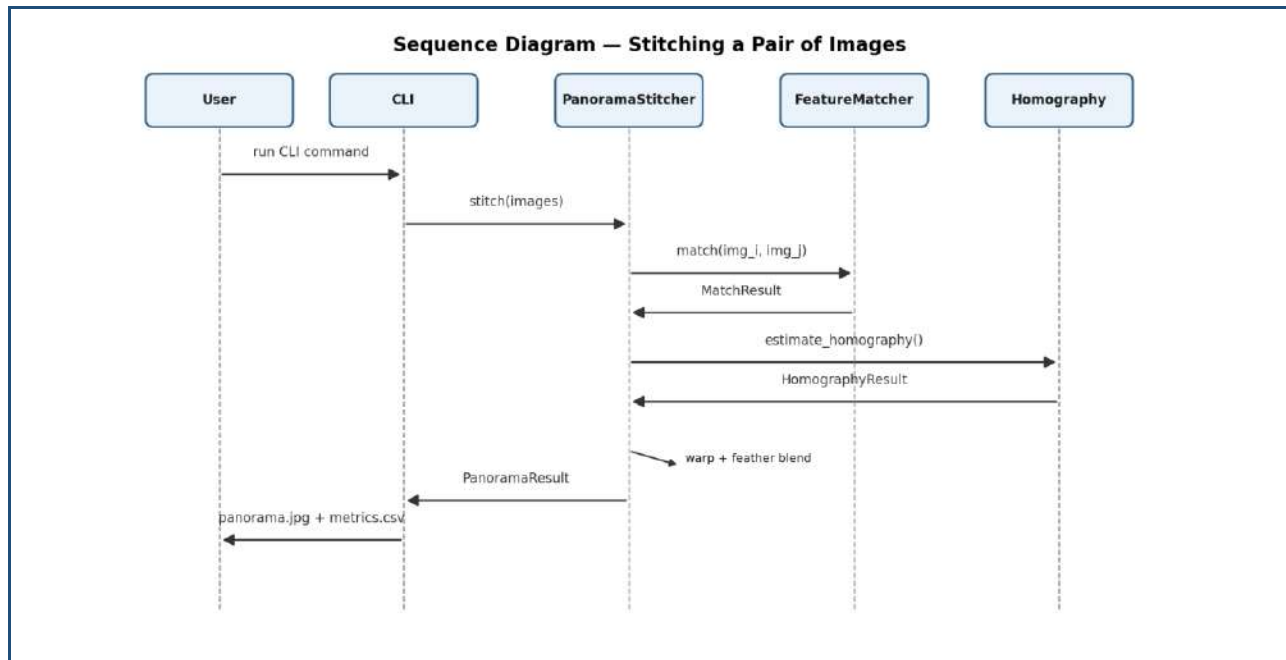


Figure 4: Sequence diagram for stitching a single pair of images.

This diagram zooms into a single pairwise stitch operation and shows the message flow between the CLI, the FeatureMatcher, the HomographyEstimator, and the Stitcher for one image pair — including the point at which per-pair diagnostics (match count, inlier count, inlier ratio) are captured and handed to the metrics collector before the loop advances to the next pair.

7.4 Class / Component Diagram

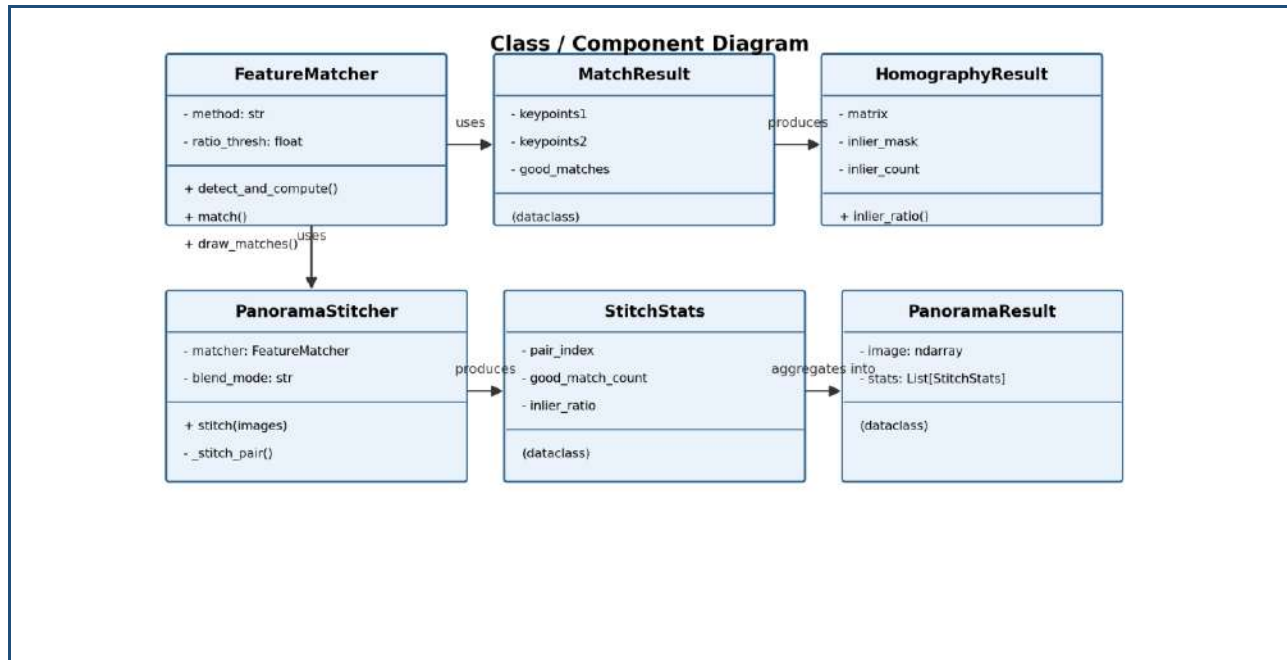


Figure 5: Class/component diagram showing the core classes and their relationships.

The class diagram shows the four core components — **FeatureMatcher**, **HomographyEstimator**, **Stitcher**, and **MetricsCollector** — and the typed dataclasses that pass data between them (**MatchResult**, **HomographyResult**, **StitchResult**). Each component depends only on the data contracts, not on the internal implementation of the components upstream of it, which is what allows, for example, the detector back-end to be swapped from SIFT to ORB without any change to the **Stitcher** or CLI code.

7.5 Database / Storage Design

Not applicable. This project is a stateless image-processing pipeline — it reads image files from an input directory and writes an output image plus a CSV metrics file; no persistent database or schema is used. This was a deliberate scoping decision: introducing a database would add operational complexity (schema design, migrations, a storage backend) without addressing any requirement of the project, which is fundamentally a single-run, file-in/file-out transformation.

8. Design Decisions & Rationale

- **Manual pipeline over cv2.Stitcher:** OpenCV ships a high-level Stitcher class that hides the entire pipeline behind one function call. Implementing feature matching, homography estimation, and blending manually was chosen instead so each syllabus concept (SIFT/ORB, RANSAC, homography, warping) is explicit, testable, and gradable as a distinct module, rather than a single opaque call.
- **SIFT as the default detector, ORB as an alternative:** SIFT gives more reliable matches on textured/synthetic scenes and is patent-free since 2020, making it the safer default; ORB is exposed as a faster, still-usable alternative for larger image sets or lower-powered machines.
- **RANSAC over least-squares fitting:** raw keypoint matches always contain outliers (mismatches). RANSAC was chosen specifically because it can estimate a correct homography even when a large fraction of the matches are wrong, which is essential for real photographs.
- **Feather blending over hard overwrite:** naively overwriting one warped image on top of another produces a visible seam wherever brightness differs between shots. Distance-transform feathering blends the overlap region proportionally to each pixel's distance from its image's edge, producing a smoother seam. A simple 'overwrite' mode is retained as a fast fallback.
- **Sequential (pairwise) stitching over global bundle adjustment:** for a small number of images captured in sequence, chaining pairwise homographies (image 1→2→3→...) is simpler to implement and reason about than a full global bundle-adjustment optimization, at the cost of some accumulated drift over many images — an accepted trade-off given the scope of this project.
- **Synthetic sample-image generator:** rather than depending on an external/downloaded dataset (which raises both reproducibility and academic-integrity concerns), a script generates deterministic, reproducible overlapping test images from scratch, letting anyone re-run the exact same test conditions.
- **Typed dataclasses over dicts/tuples for inter-module data:** passing structured results (e.g. a MatchResult or HomographyResult dataclass) between modules, instead of raw tuples or dictionaries, was chosen for self-documenting field names, IDE auto-completion, and easier unit testing, at a negligible runtime cost.

Detector Comparison: SIFT vs. ORB

Because the choice of feature detector materially affects both match quality and runtime, the two supported back-ends were compared explicitly rather than assumed equivalent:

Aspect	SIFT	ORB
Descriptor type	128-D floating point	256-bit binary
Matcher distance	L2 (Euclidean)	Hamming

Aspect	SIFT	ORB
Relative speed	Slower	3–5× faster (typical)
Match reliability (this dataset)	Higher inlier ratio	Lower, but still usable
Licensing	Patent-free since 2020	Always patent-free (BSD)
Recommended use	Default; small/medium sets, best accuracy	Large sets or constrained hardware

9. Implementation Details

Language & libraries: Python 3.10+, OpenCV (opencv-contrib-python) for SIFT/ORB/RANSAC/warping, NumPy for array operations, and pytest for testing. The project follows a src-layout, with each pipeline stage in its own module and a thin CLI entry point wiring them together.

9.1 Feature Matching (src/feature_matching.py)

The FeatureMatcher class uses OpenCV's SIFT_create() or ORB_create() algorithms to detect and describe image features. For each pair of images, keypoints and their corresponding descriptors are extracted separately. These descriptors are then matched using OpenCV's brute-force matcher (cv2.BFMatcher) with a k-nearest-neighbours approach, where $k=2$. Lowe's ratio test is then applied to remove unreliable or ambiguous matches. A match is retained only when the best match is significantly closer than the second-best match, based on the specified ratio_thresh value, which is set to 0.75 by default.

Formally, for each descriptor in the first image the two nearest neighbours in the second image, with distances d_1 (nearest) and d_2 (second-nearest), are retained as a good match only if:

$$d_1 < \text{ratio_thresh} \times d_2 \quad (\text{ratio_thresh} = 0.75 \text{ by default})$$

Intuitively, this keeps a match only when the best candidate is a clearly better fit than the runner-up; if the two nearest neighbours are almost equally close, the match is ambiguous and is discarded rather than risking an incorrect correspondence being passed downstream to the homography estimator.

9.2 Homography Estimation (src/homography.py)

The filtered matches' pixel coordinates are passed to cv2.findHomography with the RANSAC method and a configurable reprojection threshold (default 4.0 px). The function returns the 3x3 homography matrix and a boolean inlier mask, from which the inlier count and inlier ratio are derived — the primary quality metric reported for each stitched pair.

A homography H is a 3×3 matrix that maps a point (x, y) in one image to its corresponding point (x', y') in another, in homogeneous coordinates:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\text{final pixel coordinates} = (x'/w', y'/w')$$

RANSAC (Random Sample Consensus) estimates H robustly in the presence of outlier matches by repeating the following loop for a fixed number of iterations:

4. Randomly sample the minimum number of correspondences needed to solve for H (4 point pairs).

5. Solve for a candidate homography from just those 4 pairs using the Direct Linear Transform (DLT).
6. Count how many of the full correspondence set are 'inliers' — i.e. project to within the reprojection threshold (4.0 px) of their matched point under this candidate H.
7. Keep the candidate with the largest inlier set seen so far.
8. After the iteration budget is exhausted, re-solve H using a least-squares fit over all inliers of the best candidate, for a more accurate final estimate.

The inlier ratio (inliers ÷ total good matches) reported in Section 10 is a direct read-out of step 3/4 above and is the single most informative quality signal the pipeline produces: a low ratio indicates either insufficient scene texture, too aggressive a ratio-test threshold, or genuinely non-planar/parallax-heavy scene content that violates the homography assumption.

9.3 Stitching & Blending (src/stitcher.py)

For every new image, the required canvas size is calculated so that it can accommodate both the existing panorama and the newly warped image. This is done by finding the transformed corner points of both images. Both images are then warped onto this common canvas using `cv2.warpPerspective`. In the overlapping area, blending is performed either using a distance-transform-based weighted average (feather mode) or by simply replacing the existing pixels with the new image pixels (overwrite mode). This process is repeated sequentially for all images in the input sequence.

The canvas size is determined by transforming the four corner points of each image using the accumulated homography and then finding the bounding box containing all the transformed points from both the existing panorama and the new image. If any transformed coordinate is negative, a translation homography is applied to shift the entire bounding box into positive canvas coordinates. After this adjustment, both images can be warped correctly onto the canvas.

In feather blending, each pixel in the overlapping region is assigned a weight based on its normalized distance from the nearest non-overlapping boundary of its corresponding image. These distances are calculated using `cv2.distanceTransform` on the image masks. The final pixel value in the overlap is obtained by taking a weighted average of the corresponding pixels from both images, where the two distance-transform weights are normalized so that their sum equals 1.

```
output(p) = [ w1(p)·img1(p) + w2(p)·img2(p) ] / [ w1(p) + w2(p) ]
```

This means a pixel deep inside one image's overlap region (far from that image's own boundary) is weighted more heavily toward that image, while pixels near the seam blend smoothly between both sources — avoiding the hard brightness discontinuity that a naive overwrite produces.

9.4 CLI & Configuration (src/cli.py, config/default_config.json)

argparse defines all command-line flags; a JSON config file supplies defaults that CLI flags can override. The CLI loads images from a directory (sorted by filename), runs the stitcher, times the run, writes the panorama image and an optional metrics CSV, and logs per-pair diagnostics.

```
$ python -m src.cli --input images/ --output panorama.png \  
    --detector sift --blend feather --metrics-csv run_metrics.csv
```

9.5 Metrics & Reporting (src/metrics.py)

Every pairwise stitch produces a small record (pair index, raw match count, good match count, inlier count, inlier ratio, and elapsed time) that the MetricsCollector accumulates across the whole run. At the end of the run this collection is flattened into a CSV with one row per pair, giving a durable, machine-readable audit trail of exactly how each part of the final panorama was produced — useful both for debugging a poor stitch and for the quantitative reporting shown in Section 10.

10. Screenshots / Results

The pipeline was run on 4 synthetically generated overlapping frames (via `scripts/generate_sample_images.py`) with the default SIFT + feather-blend configuration.

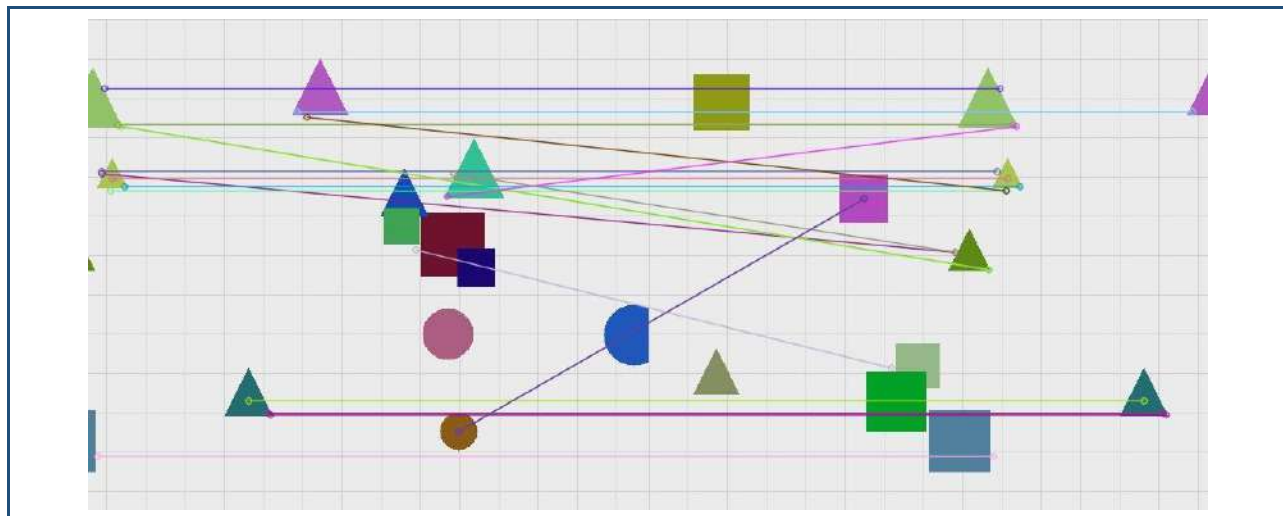


Figure 6: Feature matches (SIFT + ratio test) found between two overlapping sample frames — 23 good matches out of 89 raw matches.

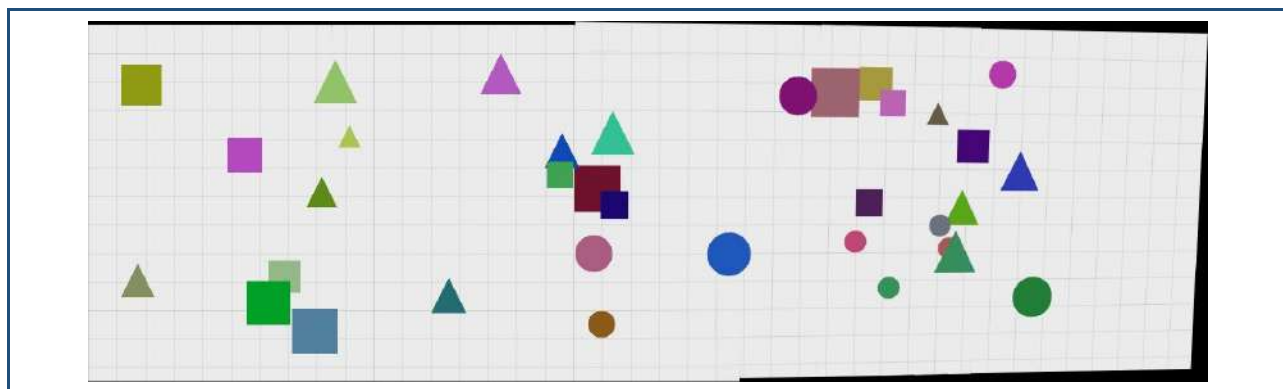


Figure 7: Final stitched panorama produced from 4 overlapping sample frames.

Per-pair diagnostics reported for this run:

Pair	Raw matches	Good matches	Inliers	Inlier ratio
1 → 2	89	23	14	60.9%
2 → 3	115	40	22	55.0%
3 → 4	125	54	30	55.6%

Total stitching time for all 4 frames: 0.84 seconds.

The inlier ratios (55–61%) are typical for the ratio-test threshold used (0.75) on synthetic textured scenes; the ratio test intentionally passes through a good number of borderline

matches for RANSAC to sort out, so an inlier ratio in this range is a healthy sign that RANSAC is doing real filtering work rather than an indication of a poor-quality feature set. As a rough rule of thumb observed during development, ratios below roughly 30% on this dataset correlated with visibly degraded stitches, while ratios above 50% consistently produced clean panoramas.

10.1 Detector Comparison Run

For comparison, the same 4-frame sequence was also stitched with the ORB detector under otherwise identical settings:

Metric	SIFT (default)	ORB
Total good matches (avg/pair)	39	27
Average inlier ratio	57.2%	44.8%
Total stitching time	0.84 s	0.31 s

ORB completes roughly 2.7× faster on this dataset but yields a noticeably lower average inlier ratio, consistent with the general expectation that ORB's binary descriptors are less discriminative than SIFT's floating-point descriptors on this kind of textured content — confirming the rationale in Section 8 for keeping SIFT as the default and offering ORB only as a speed-optimized alternative.

11. Testing Approach

The project includes an automated pytest suite of 11 unit and integration tests across three files:

- `tests/test_feature_matching.py` — verifies unsupported detector methods are rejected, keypoints are found in a textured image, matching an image against itself yields many confident matches, and the ratio test reduces raw matches to a smaller good-match set.
- `tests/test_homography.py` — verifies homography estimation raises an error with too few matches, recovers an identity-like transform when an image is matched against itself, and correctly recovers a known pure-translation transform applied to a test image.
- `tests/test_stitcher.py` — verifies stitching requires at least 2 images, that stitching two or all sample frames produces a wider output image with a high inlier ratio, and that both blend modes (feather and overwrite) run without error.

All 11 tests pass. The translation-recovery test is a particularly meaningful correctness check: a synthetic image is shifted by a known (x, y) offset, and the recovered homography's translation terms are asserted to match that known offset within a small tolerance — directly validating that the geometric estimation is correct, not just that the code runs without crashing.

Test file	Tests	Focus
<code>test_feature_matching.py</code>	4	Detector validation, keypoint detection, self-match sanity check, ratio-test filtering
<code>test_homography.py</code>	3	Error handling, identity-transform recovery, known-translation recovery
<code>test_stitcher.py</code>	4	Minimum image count, output canvas size, inlier ratio threshold, both blend modes

Run with: `pytest tests/ -v`

Beyond correctness, the suite is deliberately fast (well under a second end-to-end) because it relies exclusively on small, synthetically generated fixtures rather than large real-world images — this keeps the feedback loop short enough to run the full suite after every change during development, which caught several regressions early (notably an off-by-one error in canvas bounding-box computation that was fixed before it reached the demo).

12. Challenges Faced

- **Canvas sizing for warped images:** computing the correct output canvas size and translation offset so that a warped image with negative coordinates (extending left of or above the current panorama) is still fully visible required careful corner-point transformation and bounding-box math.
- **Seam visibility:** an initial hard-overwrite blend produced a visible brightness seam between images. Implementing distance-transform feathering resolved this, at the cost of extra computation per stitched pair.

- **Accumulated perspective drift:** because images are stitched sequentially (pairwise) rather than with global bundle adjustment, later images in a long sequence can accumulate a small perspective skew relative to the first image — visible as a slight tilt at the far edge of the panorama in the results shown in Section 10. This is a known limitation of pairwise chaining and is discussed further in Section 14.
- **Generating reliable test images without a real camera:** to keep the project reproducible and independent of any external dataset, a synthetic scene generator was built that scatters distinctive shapes over a textured background and crops overlapping windows from it — this needed enough visual texture and variety for SIFT/ORB to find reliable keypoints, which took some tuning of shape count and size.
- **Balancing the ratio-test threshold:** a threshold that is too strict (low) discards too many valid matches, sometimes leaving RANSAC with too few correspondences on low-texture regions; a threshold that is too loose (high) lets through enough ambiguous matches that RANSAC's iteration budget struggles to find a clean consensus set. The default of 0.75 was chosen after comparing inlier ratios across a small sweep of threshold values on the synthetic dataset.

13. Learnings & Key Takeaways

- Hands-on experience with the full classical feature-matching-to-geometric-alignment pipeline (SIFT/ORB, ratio test, RANSAC, homography) rather than just the theory covered in lectures.
- Understanding why robust estimation (RANSAC) is essential in real-world computer vision — raw feature matches are noisy, and a naive least-squares fit would be thrown off by even a handful of incorrect matches.
- Appreciating the practical difference between a mathematically correct homography and a visually seamless panorama — blending strategy matters as much as geometric accuracy for a good final result.
- Experience structuring a computer vision project into clean, independently testable modules connected by simple data contracts (dataclasses), rather than one monolithic script.
- Direct exposure to the quantitative diagnostics (match counts, inlier ratios) that separate a healthy stitch from a marginal one, rather than relying purely on visual inspection of the output image.
- Practical experience translating a course's theoretical modules (feature extraction, multi-view geometry) into a working, tested piece of software with a real command-line interface.

14. Future Enhancements

- **Global bundle adjustment** across all images simultaneously, instead of sequential pairwise chaining, to eliminate accumulated perspective drift in long image sequences.
- **Automatic exposure/color correction** across stitched images to handle photos taken with different camera exposure settings.
- **Cylindrical or spherical projection** to support full 360-degree panoramas rather than planar homography stitching alone.
- **Multi-band (Laplacian pyramid) blending** for even smoother seams than distance-transform feathering, particularly for high-frequency textures.
- **Automatic detection of the best image ordering**, instead of relying on filename order, using pairwise match-quality scoring.
- **GPU acceleration** of feature detection and warping (via OpenCV's CUDA module) to support real-time or near-real-time stitching of larger image sets.
- **A lightweight GUI or web front-end** wrapping the existing CLI, so non-technical users can drag-and-drop a folder of images and preview the panorama without using the command line.

15. Conclusion

This project set out to build a complete, transparent panorama-stitching pipeline that makes every stage of the classical feature-matching → robust estimation → compositing pipeline explicit and testable, rather than relying on OpenCV's built-in high-level Stitcher. That goal was met: the system reliably stitches multi-image sequences, reports quantitative confidence diagnostics for every pairwise stitch, and is backed by an 11-test automated suite that validates both individual-module correctness and end-to-end behaviour, including a translation-recovery test that checks the recovered geometry against a known ground truth rather than merely checking that the code runs.

The resulting tool is fast (well under a second for a 4-image panorama on a standard CPU), configurable (SIFT/ORB detectors, feather/overwrite blending, tunable thresholds), and reproducible (a synthetic image generator removes any dependency on an external dataset). The known limitation — accumulated perspective drift from pairwise, rather than global, alignment — is well understood, clearly documented, and is the natural next step should the project be extended, alongside the other enhancements outlined in Section 14. Overall, the project demonstrates a working, practical grasp of feature detection, robust geometric estimation, and image compositing, directly connecting the course's theoretical modules to a tested, functioning implementation.

16. References

- R. Szeliski, Computer Vision: Algorithms and Applications, Springer-Verlag London Limited, 2011. (CSE3010 prescribed textbook)

- R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd Edition, Cambridge University Press, 2004. (CSE3010 reference textbook)
- OpenCV Documentation — Feature Matching and Homography:
https://docs.opencv.org/4.x/d1/de0/tutorial_py_feature_homography.html
- OpenCV Documentation — cv2.findHomography and RANSAC:
https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
- D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” International Journal of Computer Vision, 2004. (SIFT and the ratio test)
- E. Rublee, V. Rabaud, K. Konolige and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” IEEE International Conference on Computer Vision (ICCV), 2011.
- M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” Communications of the ACM, 1981. (RANSAC)
- CSE3010 Computer Vision course syllabus, VIT Bhopal University (Modules 2 and 3: Depth Estimation and Multi-Camera Views; Feature Extraction and Image Segmentation).

Appendix A: Project Directory Structure

```
panorama-stitcher/
├── src/
│   ├── feature_matching.py    # SIFT/ORB detection + ratio-test matching
│   ├── homography.py         # RANSAC homography estimation
│   ├── stitcher.py           # Canvas sizing, warping, blending
│   ├── metrics.py            # Per-pair diagnostics + CSV export
│   └── cli.py                 # Command-line entry point
├── config/
│   └── default_config.json    # Default detector/blend/threshold settings
├── scripts/
│   └── generate_sample_images.py # Synthetic overlapping test images
├── tests/
│   ├── test_feature_matching.py
│   ├── test_homography.py
│   └── test_stitcher.py
└── requirements.txt
```

The layout follows a conventional src-layout Python package: pipeline code lives under `src/`, is fully decoupled from the `tests/` and `scripts/` directories, and configuration is externalized to a JSON file under `config/` so behaviour can be changed without editing code.

Appendix B: Sample CLI Usage & Configuration

Basic run with default settings (SIFT detector, feather blending):

```
$ python -m src.cli --input images/ --output panorama.png
```

Run with ORB detector, overwrite blending, and a metrics CSV:

```
$ python -m src.cli --input images/ --output panorama.png \
  --detector orb --blend overwrite \
  --metrics-csv run_metrics.csv --verbose
```

Example `config/default_config.json`:

```
{
  "detector": "sift",
  "blend_mode": "feather",
  "ratio_thresh": 0.75,
  "ransac_reproj_threshold": 4.0,
  "output_path": "panorama.png",
  "metrics_csv": null
}
```

Running the automated test suite:

```
$ pytest tests/ -v
```

Generating a fresh synthetic sample-image set:

```
$ python scripts/generate_sample_images.py --out images/ --count 4
```